This ICCV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Any-SSR: How Recursive Least Squares Works in Continual Learning of Large Language Models

Kai Tong[1], **Kang Pan**[2], **Xiao Zhang**[2], **Erli Meng**[2], **Run He**[1], **Yawen Cui**[3], **Nuoyan Guo**[1], **Huiping Zhuang**[1*]
[1]South China University of Technology [2]Xiaomi Corporation
[3]The Hong Kong Polytechnic University

Abstract

Fine-tuning large language models (LLMs) on data of various domains encompassing advanced language-related tasks. Such a process is essentially a continual learning procedure since it is impractical to re-access the pre-trained data in most parties. This naturally attracts catastrophic forgetting of LLMs' previously learned knowledge, such as general skills. Existing techniques either leverage previous data to replay, leading to extra computational costs, or utilize a single parameter-efficient module to learn the downstream task, constraining new knowledge absorption with interference among different tasks. To handle these issues, we propose an Analytic Subspace Routing (Any-SSR) to address these challenges. For each task, we isolate the learning within a subspace of deep layers' features via low-rank adaptation, eliminating knowledge interference between different tasks. Additionally, we propose an analytic routing mechanism to properly utilize knowledge learned in different subspaces. Our approach employs Recursive Least Squares to train a multi-task router model, allowing the router to dynamically adapt to incoming data without requiring access to historical data. Subsequently, the router effectively assigns the current task to an appropriate subspace and has a non-forgetting property of previously learned tasks with a solid theoretical guarantee. Experimental results demonstrate that our method achieves near-perfect retention of prior knowledge while seamlessly integrating new information, effectively mitigating the core limitations of existing methods. Our code is available at https://github.com/ZHUANGHP/Any-SSR.

1. Introduction

Recent years have witnessed remarkable advances in large language models (LLMs) [6, 27, 36], which have emerged as foundational technologies driving breakthroughs across diverse tasks, including natural language understanding [17, 49], multimodal reasoning [10, 45], and application in robotics [5, 22]. However, a critical challenge persists in dynamic real-world environments, where new concepts and domains of knowledge emerge and vary frequently. To maintain LLMs' efficacy, models are required to absorb new domain knowledge through a fine-tuning (FT) process. Such a process essentially equals to continual learning (CL) of LLMs, with an aim to retain previously learned knowledge while accepting new-domain data.

A main unsolved problem in CL is the issue of *catastrophic forgetting* [1], where the models could quickly lose the previous learned knowledge when accepting new tasks. This phenomenon is particularly severe in LLMs, since the general capabilities of LLMs encoded within high-dimensional parameter spaces can be easily disrupted by FT. This causes both prior task-specific knowledge and core generative competencies [42]. Unlike traditional CL problems, prolonged iteration process in LLMs' continual fine-tuning induces cumulative parameter drift. This could result in a severe decline of LLMs' task-specific abilities, as well as a significant portion of their general knowledge.

A traditional strategy to counter catastrophic forgetting is to store a small set of used data as exemplars for replaying [13, 26]. However, replaying data for LLMs is less practical due to the huge volume of training data (especially those from pre-training). Moreover, replaying stored data could raise intensive computational cost and accompany potential privacy invasion. To avoid this, several techniques utilize parameter-efficient tuning (PET) [12] including prompt tuning and low-rank adaptation (LoRA) [16] to continually adapt LLMs to new tasks. These methods mainly use lightweight module with a fixed number of parameters to absorb knowledge from all downstream tasks [30, 38]. Fine-tuning the shared module on one task followed by another consecutively induces catastrophic forgetting due to an extreme imbalanced data distribution [53]. It is hence highly motivated to explore new LLM technique that can continually absorb new skills in a replay-free fashion while overcoming

*Corresponding author (e-mail: hpzhuang@scut.edu.cn)

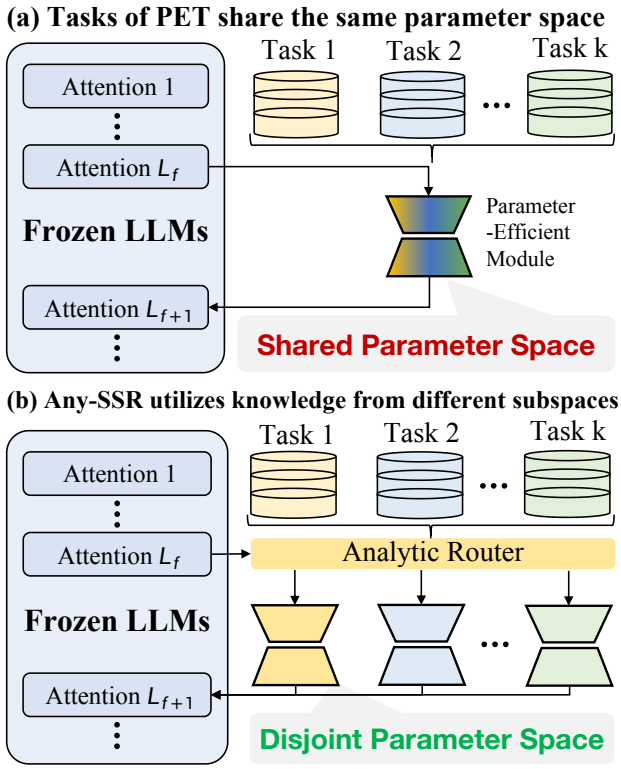


Figure 1. The difference between our method and parameter-efficient tuning (PET)-based approaches. While traditional fine-tuning introduces cumulative parameter drift that erodes both task-specific skills and core capabilities, our approach employs disjoint parameter spaces for individual tasks to avoid forgetting.

the nature of catastrophic forgetting.

In this paper, we propose an Analytic Subspace Routing (Any-SSR) framework for continual learning of LLMs. Unlike common LLMs-CL techniques, the proposed Any-SSR creatively adopts a classic recursive least squares (RLS) closed-form solution as an important mean to battle catastrophic forgetting. This comes in a hybrid structure, i.e., with an analytic router branching out in an earlier frozen segment of LLM layers, and the remaining layers tuned by LoRA using normal gradient-based technique. For each task, we isolate the learning within a subspace of deep layers' features via low-rank adaptation, eliminating knowledge interference among different tasks. We trained a specialized LoRA attachment, and use the analytic router to tell which one to load when inference. The key to avoid forgetting is that the analytic router bears the equivalence between continual training one task at a time, and the joint training by putting all-the-task data simultaneously. The main contributions of this work are listed as follow:

- We introduce the Any-SSR, a novel closed-form subspace routing approach for continual learning in LLMs, facilitating dynamic and adaptable learning of new tasks.
- Our method structure each task as a subspace training using a specific LoRA model attachment, thereby reducing potential knowledge interference among tasks.
- Our method includes an analytic routing mechanism derived through recursive least squares, which seamlessly incorporate new tasks without forgetting. A theoretical guarantee of this non-forgetting property is provided to affirm its efficacy in mitigating catastrophic forgetting.
- We validate the Any-SSR through comprehensive experiments, achieving state-of-the-art performance on the Trace metric [39].

2. Related Works

2.1. Traditional Continual Learning

Traditional continual learning aims at continuously updating models with data streams from scratch while addressing catastrophic forgetting during the process. Existing attempts on traditional continual learning can be divided into several categories. Replay-based methods [3, 14, 32, 37, 47] mitigate catastrophic forgetting by reintroducing part of past samples as exemplars to replay during training with several variants enhancing performance or optimizing memory usage. Regularization-based methods [18, 19, 21, 46] impose additional constraints on network activations or key parameters to mitigate forgetting. Prototype-based methods [20, 28, 34, 43, 48] employ prototypes to preserve decision boundaries across old and new tasks in the perspective of classifier training. Analytic learning-based methods [50–53] replace traditional backpropagation with a closed-form solution using techniques like Regularized Least Squares to achieve identical results of continual learning as those obtained by joint learning. However, implementing these techniques in LLMs poses several challenges. For instance, storing past data in replay-based methods can result in substantial computational expenses, determining parameter importance in regularization-based methods continues to be challenging due to the vast scale of LLMs, and prototype-based and analytic learning-based methods are oriented towards classifier learning, a component not present in LLMs. These methods also struggle with preserving general knowledge in LLMs post large-scale pretraining. To address these issues, we introduce Analytic Subspace Routing for continual learning in LLMs.

2.2. Continual Learning of LLMs

Due to the extensive scale of parameters in LLMs and the general skills acquired through intensive pre-training, CL in LLMs introduces additional challenges related to training costs and general knowledge retention. While various approaches have attempted to adapt traditional CL techniques to mitigate general knowledge forgetting, they are often hindered by the high training costs or the need to store previous

data [13, 33]. For instance, SEEKR [13] employs a method that involves the selection of attention heads based on forgettability and task-sensitivity metrics, followed by attention distillation on these heads using stored samples. Despite its ability to reduce the dependence on extensive data replaying, the complete fine-tuning process in SEEKR can still be costly. Various techniques have explored parameter-efficient tuning to alleviate training costs [8, 15]. For example, O-LoRA focuses on LoRA-based architectures and facilitates continual learning within orthogonal subspaces [15]. Nevertheless, these methods typically rely on a fixed number of parameters to assimilate knowledge from subsequent tasks, resulting in ineffective adaptation to new tasks and potential interference between different tasks [29]. To tackle the challenges associated with high training costs and potential knowledge interference across tasks, we propose the Analytic Subspace Routing technique, which leverages parameter-efficient strategies and isolates subspaces to acquire distinct knowledge effectively.

3. The Proposed Method

3.1. Problem Definition

The primary objective of CL for LLMs lies in optimizing their performance on sequentially acquired data streams while maintaining minimal degradation of their foundational competencies. Suppose a sequence of new tasks denoted as $\{\mathcal{D}_1, \ldots, \mathcal{D}_k\}$ arrives in a flow-like fashion, each task $\mathcal{D}_k = \{(\boldsymbol{x}^i, \boldsymbol{y}^i)\}_{i=1}^{n_k}$ contains n_k pairs of question-answer($\boldsymbol{x}$, $\boldsymbol{y}$) data. A single model needs to adapt to them sequentially with only access to $\mathcal{D}_k$ at the k-th task. In general, given a prediction model M_θ parameterized by θ, continual learning seeks to optimize for the following log-likelihood across all tasks:

$$\max_{\theta} \sum_{i=1}^{k} \sum_{\boldsymbol{x},\boldsymbol{y}\in\mathcal{D}_i} \log p_\theta(\boldsymbol{y}|\boldsymbol{x}). \tag{1}$$

Our approach incorporates the concept of recursion. In the following text, we first solve the scenarios with data $\{\mathcal{D}_1, \ldots, \mathcal{D}_k\}$ using base training. Subsequently, we will illustrate how to recursively solve the problem when $\mathcal{D}_{k+1}$ arrives without leveraging the source data from $\mathcal{D}_1$ to $\mathcal{D}_k$.

3.2. Overall Architecture

We propose a closed-form LLM continual learning method, which achieves organic unification of shared representation and task-specific specialization through structural disentanglement. As shown in 2, the architecture comprises three core components: Frozen universal feature extractor, Task-specific Low-Rank Adaptation (LoRA) [16] bank, and Recursive Analytic Learning(AL) based task router. For an input sequence the feedforward process is formalized as:

$$\boldsymbol{y}_{t+1} = h_{\leq L_f}(\boldsymbol{x}) \cdot f_{\theta_{k^*}}(h_{>L_f}(\boldsymbol{x})), \tag{2}$$

where $k^* = \text{argmax}_k\, g_k(h_{\leq L_f}(\boldsymbol{x}))$ indicates the selected task-id of the router, $h_{\leq L_f}$ denotes the frozen parameters of LLM's first L_f layers, $h_{>L_f}$ represents the the base parameters of subsequent layers, f_{θ_k} is the LoRA adapter for task k, g_k denotes the normalized weights generated by the routing function, $\boldsymbol{x}$ is the input sequence of length t, and $\boldsymbol{y}_{t+1}$ is the predicted next word. This design enables dynamic combination of base model capacity sharing and task-specific parameterization.

3.3. Hierarchical Feature Disentanglement

Inspired by hierarchical processing in cortical systems, we hypothesize that lower transformer layers encode cross-task semantic features while higher layers handle task-specific semantic composition. Based on this, we partition parameters of the pre-trained LLM:

$$h_{\leq L_f} = \bigotimes_{l=1}^{L_f} \text{TransformerBlock}_l \tag{3}$$

$$h_{>L_f} = \bigotimes_{l=L_f+1}^{L_{\text{total}}} \text{TransformerBlock}_l. \tag{4}$$

Here, $h_{\leq L_f}(\boldsymbol{x}) \in \mathbb{R}^{t\times d}$ denotes the feature extracted by the low layers, t denotes the length of the input $\boldsymbol{x}$, d is the hidden size of the transformer layers, $\bigotimes$ represents layer-wise composition, and TransformerBlock indicates the frozen parameters of layer l. The benefit of disentanglement is that frozen lower layers preserve general linguistic understanding from pretraining.

For each new task $\mathcal{D}_k$, we insert a unique trainable LoRA adapter into shared higher layers. That is, given weight matrix $\boldsymbol{W}_l^{(k)}$ in layer l, LoRA decomposes parameter updates via low-rank projection:

$$\Delta\boldsymbol{W}_l^{(k)} = \boldsymbol{B}_l^{(k)}\boldsymbol{A}_l^{(k)},\ \boldsymbol{B}_l^{(k)} \in \mathbb{R}^{d_{\text{in}}\times r},\ \boldsymbol{A}_l^{(k)} \in \mathbb{R}^{r\times d_{\text{out}}}, \tag{5}$$

where $r \ll \min(d_{\text{in}}, d_{\text{out}})$ is the rank hyperparameter. The forward computation for a sequence $\boldsymbol{x}$ becomes:

$$f_{\theta_k}(\boldsymbol{h}) = h_{>L_f}(\boldsymbol{x}) + \sum_{l=L_f+1}^{L_{\text{total}}} (\Delta\boldsymbol{W}_l^{(k)}\boldsymbol{h_l}). \tag{6}$$

Unlike conventional continual learning, all task adapters are trained independently with frozen base parameters to prevent interference.

3.4. Analytic Routing Mechanism

The router needs to process low-layer features $h_{\leq L_f}(\boldsymbol{x})$ to predict task affiliation distribution $p(k|\boldsymbol{x})$, and update the weights without history data when new tasks are introduced. We propose a forgetting-free routing strategy based on ridge

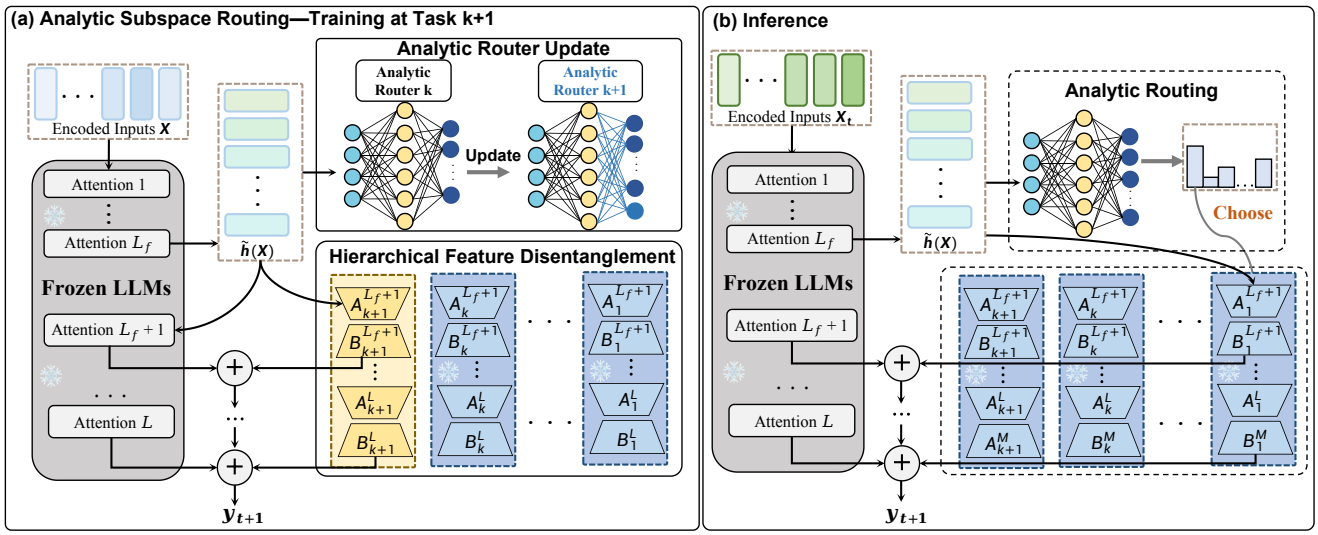


Figure 2. An overview of our proposed method Any-SSR. All the original parameters of the LLM are frozen. (a) The input data enters the first L_f layers of the Transformer blocks in the model, and the feature $h_{\leq L_f}(\boldsymbol{x})$ are obtained. Features will be expanded to a larger dimension, then an analytic router will be trained in the manner of ridge regression. The trained router will perform recursive updates based on AutoCor $\boldsymbol{R}_k$ (or $\boldsymbol{R}_k$ and $\boldsymbol{Q}_k$) and the new coming dataset.(b) The inference module. The trained router accept the feature as input, and is responsible for selecting which LoRA task model to use during the inference process.

regression, which consists of two core components: **Feature Expansion Module** that enhances feature separability and **Recursive Least-Square Solution**, enabling incremental task learning.

Given frozen features $h_{\leq L_f}(\boldsymbol{X})$ (where $\boldsymbol{X}$ denotes all the samples from task 1 to k), we project them to an expanded space:

$$\tilde{\boldsymbol{h}} = \phi(\text{mean-pool}(h_{\leq L_f}(\boldsymbol{X}))) \in \mathbb{R}^E. \tag{7}$$

The mean-pool is used to eliminate the dimension of the sequence length, that is:

$$\text{mean-pool}(\boldsymbol{H}) = \frac{1}{T}\sum_{t=1}^{T}\boldsymbol{H}_t, \quad \boldsymbol{H} \in \mathbb{R}^{T\times d}, \tag{8}$$

where $\boldsymbol{H} = h_{\leq L_f}(\boldsymbol{X})$ denotes the hidden states, $\phi : \mathbb{R}^d \to \mathbb{R}^E$ is a Gaussian-initialized, weight-frozen projection, and it is implemented via linear transformation with ReLU activation ($E > d$ as hyperparameter). The underlying cause is that expanding nonlinear feature expansion into higher-dimensional spaces exponentially improves linear separability probability, which is demonstrated by Cover's theorem [7].

The calculation of the analytic router is a linear regression problem, the goal is to minimize:

$$\underset{\boldsymbol{W}_\text{r}}{\text{argmin}}(||\boldsymbol{Y}_{1:k} - \tilde{\boldsymbol{h}}_{1:k}\boldsymbol{W}_k^\text{r}||_\text{F}^2 + \lambda||\boldsymbol{W}_k^\text{r}||_\text{F}^2), \tag{9}$$

where $\lambda > 0$ is the coefficient of the regularization term, $\boldsymbol{W}_k^\text{r} \in \mathbb{R}^{E\times k}$ indicates the weight of the analytic router we aim to calculate and

$$\tilde{\boldsymbol{h}}_{1:k} = \begin{bmatrix}\tilde{\boldsymbol{h}}_1\\ \tilde{\boldsymbol{h}}_2\\ \vdots\\ \tilde{\boldsymbol{h}}_k\end{bmatrix}, \qquad \boldsymbol{Y}_{1:k} = \begin{bmatrix}\boldsymbol{Y}_1\\ \boldsymbol{Y}_2\\ \vdots\\ \boldsymbol{Y}_k\end{bmatrix}, \tag{10}$$

where $\boldsymbol{Y}_1, \ldots, \boldsymbol{Y}_k$ are the task-IDs, i.e., the label of the router. Since the problem is a convex optimization problem, its solution can be obtained by equating its gradient to zero:

$$\hat{\boldsymbol{W}}_k^\text{r} = \left(\sum_{i=1}^{k}\tilde{\boldsymbol{h}}_i^\top\tilde{\boldsymbol{h}}_i + \lambda\boldsymbol{I}\right)^{-1}\left(\sum_{i=1}^{k}\tilde{\boldsymbol{h}}_i^\top\boldsymbol{Y}_i\right), \tag{11}$$

with $\boldsymbol{I}$ denotes the identity matrix.

3.5. Recursive Incremental Update

Let $\boldsymbol{R}_k = \left(\sum_{i=1}^{k}\tilde{\boldsymbol{h}}_i^\top\tilde{\boldsymbol{h}}_i + \lambda\boldsymbol{I}\right)^{-1}$ and $\boldsymbol{Q}_k = \sum_{i=1}^{k}\tilde{\boldsymbol{h}}_i^\top\boldsymbol{Y}_i$, where $\boldsymbol{R}_k \in \mathbb{R}^{E\times E}$ is an autocorrelation matrix (AutoCor) and $\boldsymbol{Q}_k \in \mathbb{R}^{E\times k}$ is a cross correlation matrix (CrossCor), $\boldsymbol{R}_k$ and $\boldsymbol{Q}_k$ capture the correlation information of $\tilde{h}_{1:k}$ and $\boldsymbol{Y}_{1:k}$. These matrix can recursively update the router weight $\boldsymbol{W}_k^\text{r}$ when new task $k+1$ arrives without using history datasets $\{\mathcal{D}_1, \ldots, \mathcal{D}_k\}$ as follow.

Based on (11), we have

$$\hat{\boldsymbol{W}}_{k+1}^\text{r} = \boldsymbol{R}_{k+1}\boldsymbol{Q}_{k+1}. \tag{12}$$

According to the Woodbury matrix identity [41], for conformable matrices $\boldsymbol{A}$, $\boldsymbol{U}$, $\boldsymbol{C}$, and $\boldsymbol{V}$, we have

$$(\boldsymbol{A}+\boldsymbol{U}\boldsymbol{C}\boldsymbol{V})^{-1} = \boldsymbol{A}^{-1}-\boldsymbol{A}^{-1}\boldsymbol{U}(\boldsymbol{C}^{-1}+\boldsymbol{V}\boldsymbol{A}^{-1}\boldsymbol{U})^{-1}\boldsymbol{V}\boldsymbol{A}^{-1}. \tag{13}$$

Let $\boldsymbol{A} = \boldsymbol{R}_k^{-1}$, $\boldsymbol{U} = \tilde{\boldsymbol{h}}_{k+1}^\top$, $\boldsymbol{V} = \tilde{\boldsymbol{h}}_{k+1}$, and $\boldsymbol{C} = \boldsymbol{I}$, we can obtain

$$\boldsymbol{R}_{k+1} = \boldsymbol{R}_k - \boldsymbol{R}_k\tilde{\boldsymbol{h}}_{k+1}^\top(\boldsymbol{I} + \tilde{\boldsymbol{h}}_{k+1}\boldsymbol{R}_k\tilde{\boldsymbol{h}}_{k+1}^\top)^\top\tilde{\boldsymbol{h}}_{k+1}\boldsymbol{R}_k. \tag{14}$$

According to the definition of $\boldsymbol{Q}_k$, the iterative expression of $\hat{\boldsymbol{W}}_{k+1}^{\mathrm{r}}$ can be written as

$$\hat{\boldsymbol{W}}_{k+1}^{\mathrm{r}} = \boldsymbol{R}_{k+1}\boldsymbol{Q}_k + \boldsymbol{R}_{k+1}\tilde{\boldsymbol{h}}_{k+1}^\top\boldsymbol{Y}_{k+1}. \tag{15}$$

By substituting Eq. (14) into Eq. (15), we have

$$\begin{aligned}\hat{\boldsymbol{W}}_{k+1}^{\mathrm{r}} &= \boldsymbol{R}_k\boldsymbol{Q}_k + \boldsymbol{R}_k\tilde{\boldsymbol{h}}_{k+1}^\top\boldsymbol{Y}_{k+1} \\ &\quad -\boldsymbol{R}_k\tilde{\boldsymbol{h}}_{k+1}^\top(\boldsymbol{I} + \tilde{\boldsymbol{h}}_{k+1}\boldsymbol{R}_k\tilde{\boldsymbol{h}}_{k+1}^\top)^{-1}\tilde{\boldsymbol{h}}_{k+1}\boldsymbol{R}_k\boldsymbol{Q}_k \\ &\quad -\boldsymbol{R}_k\tilde{\boldsymbol{h}}_{k+1}^\top(\boldsymbol{I} + \tilde{\boldsymbol{h}}_{k+1}\boldsymbol{R}_k\tilde{\boldsymbol{h}}_{k+1}^\top)^{-1}\tilde{\boldsymbol{h}}_{k+1}\boldsymbol{R}_k\tilde{\boldsymbol{h}}_{k+1}^\top\boldsymbol{Y}_{k+1}.\end{aligned} \tag{16}$$

$\hat{\boldsymbol{W}}_{k+1}^{\mathrm{r}}$ can be calculated using the previously AutoCor $\boldsymbol{R}_k$ and CrossCor $\boldsymbol{Q}_k$, as well as the features $\tilde{\boldsymbol{h}}_{k+1}$ and labels $\boldsymbol{Y}_{k+1}$ of the newly incoming data. For simplicity, $\hat{\boldsymbol{W}}_{k+1}^{\mathrm{r}}$ can also be expressed as:

$$\hat{\boldsymbol{W}}_{k+1}^{\mathrm{r}} = (\boldsymbol{I} - \boldsymbol{R}_{k+1}\boldsymbol{X}_{k+1}^\top\boldsymbol{X}_{k+1})\hat{\boldsymbol{W}}_k^{\mathrm{r}} + \boldsymbol{R}_{k+1}\boldsymbol{X}_{k+1}^\top\boldsymbol{Y}_{k+1}. \tag{17}$$

The proof can be found in the Supplementary material A.

By maintaining $\boldsymbol{R}_k$, our method achieves replay-free continual learning of the analytic router, which incrementally absorbs new tasks without forgetting historical knowledge.

3.6. Inference Procedure

The inference process enables task-aware generation through analytic routing. Let input sequence be $\boldsymbol{X}_t = \{x_1, ..., x_t\}$, the features passing through frozen feature extractor are

$$h_{\le L_f}(\boldsymbol{X_t}) = \bigotimes_{l=1}^{L_f} \text{TransformerBlock}_l(\boldsymbol{X_t}). \tag{18}$$

Then the feature expansion follows,

$$\tilde{\boldsymbol{h}}(\boldsymbol{X_t}) = \phi(\text{mean-pool}(h_{\le L_f}(\boldsymbol{X_t}))) \in \mathbb{R}^{d'}. \tag{19}$$

Then the task affiliation probability is computed via analytic router weights,

$$p(k|\boldsymbol{X_t}) = \text{softmax}\left(\tilde{\boldsymbol{h}}(\boldsymbol{X_t})\boldsymbol{W}_k^{\mathrm{r}}\right). \tag{20}$$

Based on the probability, the dominant task expert is selected,

$$k^* = \underset{k}{\text{argmax}}\, p(k|\boldsymbol{X}_t). \tag{21}$$

Then corresponding LoRA adapter is activated, and the next predicted token y_{t+1} is obtained

$$y_{t+1} \sim f_{\theta_{k^*}}(h_{>L_f}(\boldsymbol{X}_t)). \tag{22}$$

The generated token is appended to the input sequence $\boldsymbol{X}_t$ and this process repeats until $y_{t+1} = eos_token$:

$$\boldsymbol{X}_{t+1} = \boldsymbol{X_t} \oplus \boldsymbol{y}_{t+1}. \tag{23}$$

The proposed method preserves general knowledge via frozen base layers, achieves continual learning through analytic routing and incremental LoRA expert construction. The inference phase employs feature expansion and closed-form routing for task-aware generation. Our method have such advantages: (1) Forgetting prevention via parameter disentanglement and recursive updating; (2) Compact LoRA storage per task ($\mathcal{O}(rL_{\text{adapt}})$ parameters) reduces $> 99\%$ storage vs full fine-tuning; (3) Historical data independence through AutoCor matrix storage; (4) CPU-compatible lightweight routing.

Algorithm 1 The training process of the Analytic Router

Analytic router training for $\mathcal{D}_1, \ldots, \mathcal{D}_k$.
1. Extract features $h_{\le L_f}(\boldsymbol{X})$
2. Expand feature to $\tilde{\boldsymbol{h}}(\boldsymbol{X})$ via (7)
3. **for** $i = 1$ to k **do**:
   Train LoRA model $f_{\theta_i}(\boldsymbol{h})$ via (6)
   end for
4. Obtain analytic router weight $\hat{\boldsymbol{W}}_k^{\mathrm{r}}$ via (11)
5. Save the AutoCor $\boldsymbol{R}_k$ and CrossCor $\boldsymbol{Q}_k$

Continual Router Training for $\mathcal{D}_{k+1}$
1. Extract feature $h_{\le L_f}(\boldsymbol{X}_{k+1})$
2. Train LoRA model $f_{\theta_{k+1}}(\boldsymbol{h})$ via (6)
3. Obtain analytic router weight $\hat{\boldsymbol{W}}_{k+1}^{\mathrm{r}}$ via (11)
4. Save the AutoCor $\boldsymbol{R}_{k+1}$ and CrossCor $\boldsymbol{Q}_{k+1}$

Algorithm 2 Inference Procedure

Inference($\boldsymbol{x}_t$)
while $y_{t+1} \neq$ eos_token **do**
1. Extract feature $h_{\le L_f}(\boldsymbol{x}_t)$ via $\boldsymbol{x}_t$
2. Expand feature to $\tilde{\boldsymbol{h}}(\boldsymbol{x}_t)$ via (7)
3. Calculate task probability via (20)
4. Pick dominate task LoRA adapter via (21)
5. Forward propagation: $y_{t+1} \leftarrow f_{\theta_{k^*}}(h_{>L_f}(\boldsymbol{x}_t))$
6. Update $\boldsymbol{x}_{t+1} \leftarrow \text{concat}(\boldsymbol{x}_t, \boldsymbol{y}_{t+1})$

end while

4. Experiments

4.1. Benchmarks

We evaluate our proposed method using the TRACE benchmark [39], a widely recognized framework for assessing

continuous learning in LLMs. TRACE includes eight diverse datasets that span various domains, such as domain-specific knowledge, multilingual understanding, code generation, and mathematical reasoning. For our experiments, we use the reasoning-augmented versions of these datasets and follow the two task order configurations outlined in the original study. After the continuous learning process, we assess both the performance on the newly learned tasks and any changes in the general abilities of the LLMs. We follow SEEKR [13] to employ two distinct training sequences:

Table 1. Two distinct CL task orders.

Order1:	C-STANCE → FOMC → MeetingBank → Py150 → ScienceQA → NumGLUE-cm → NumGLUE-ds → 20Minuten
Order2:	NumGLUE-cm → NumGLUE-ds → FOMC → 20Minuten → C-STANCE → Py150 → MeetingBank → ScienceQA

4.2. Baselines and Evaluation Metrics

In this paper, we employ the overall performance (OP) [4] and the backward transfer scores (BWT) [25] as the principal evaluation metrics. For tasks $\{\mathcal{D}_1, \ldots, \mathcal{D}_k\}$, upon incrementally learning the k-th task $\mathcal{D}_k$, the score achieved by the model on the i-th task $\mathcal{D}_i$ is represented as $A_{i,t}$ (where $t \geq i$). The computations for the OP and the BWT are carried out using the following formulas:

$$\text{OP} = \frac{1}{k}\sum_{i=1}^{k} A_{i,k}, \tag{24}$$

$$\text{BWT} = \frac{1}{k-1}\sum_{i=1}^{k-1}(A_{i,k} - A_{i,i}). \tag{25}$$

We compare our proposed method, Any-SSR, with ten baseline approaches commonly used in the context of continuous learning:

AdaLoRA [44]: AdaLoRA parameterizes incremental updates via singular value decomposition and adaptively allocates parameter budget among weight matrices based on their importance scores. **LoRAMoE** [9]: Introduces several LoRA adapters as experts and integrates them via a router network, with a localized balancing constraint during training to alleviate world knowledge forgetting in LLMs while enhancing downstream task performance. **P-tuning v2** [23]: Inserts trainable continuous prompts into every layer of a pre-trained LLM. **EWC** [18]: Regularizes parameter updates by constraining changes based on their importance scores. **LwF** [21]: Uses knowledge distillation, where the model for the previous task is distilled into the current task using the current task's data. **Replay**: Fine-tunes the model on the current task's data along with a small set of replay samples from previous tasks. **DER++** [2]: Stores model predictions (logs) as replay samples from previous tasks and combines knowledge distillation and replay to reduce forgetting. **LFPT5**[30]: Learns a soft prompt to generate pseudo-samples of previous tasks, enabling replay without explicitly storing past data. **O-LoRA** [38]: Applies orthogonal constraints to LoRA matrices for each task, ensuring task-specific updates while reducing interference. **L2P** [40]: Maintains a prompt pool to dynamically select and fine-tune prompts specific to individual samples. **PP** [31]: Tunes a set of prompts for each task and concatenates them during inference to encode task-specific knowledge. **SEEKR** [13]: Conducts attention distillation on the selected attention heads to recognize the most valuable attention heads.

We also include the results of **multi-task learning** (MTL) as an upper-bound reference, representing the ideal scenario where the model is trained on all tasks simultaneously [24].

4.3. Training details

Our experiments were conducted on the Llama2-7B-chat foundation model [36] using the HuggingFace Transformers implementation. To demonstrate the robustness of our method, we also conducted experiments on GLM-4-9B [11] and Qwen2.5-7B [35], and the results can be found in the Supplementary material B. These results across different foundation models remain consistent and confirm Any-SSR's generalizability across architectures. Following Trace [39], we maintained a consistent training sample size of 5,000 with a constant learning rate of 1e-5 across all tasks. The model was trained on eight benchmark datasets spanning multiple domains: C-STANCE (social media stance detection), FOMC (financial decision parsing), MeetingBank (meeting summarization), Py150 (code generation), ScienceQA (science question answering), and two numerical reasoning datasets (NumGLUE-cm, NumGLUE-ds), along with the 20Minuten news comprehension corpus. We trained the model for 10 epochs.

The optimization configuration employed AdamW with disabled weight decay and a batch size of 128. All experiments were executed on 7×NVIDIA GeForce RTX 4090 GPUs (24GB VRAM each) using DeepSpeed ZeRO-3 optimization.

4.4. Experimental Results

4.4.1. Performance on Continual Learning Tasks

To validate the performance, we compared the proposed Any-SSR with baseline methods on the TRACE benchmark using two distinct learning orders. Table 2 illustrates that Any-SSR achieves competitive performance in both OP and BWT with significant margins when compared to exemplar-free methods. For instance, when assessed with Order1, Any-SSR achieves an OP of 55.69%, surpassing the second exemplar-

Table 2. Experiments on the TRACE benchmark using LLaMA-2-7B-Chat. The results in % are generated by two different transfer orders, and are displayed in the format of OP (BWT).

	Order1	Order2
AdaLoRA	22.60 (−30.11)	23.34 (−27.54)
LoRAMoE	48.54 (−4.27)	47.48 (−4.28)
P-tuning v2	13.10 (−17.19)	13.23 (−17.23)
SeqFT	47.63 (−11.45)	45.12 (−12.27)
EWC	48.20 (−9.48)	44.54 (−12.00)
LwF	41.86 (−6.50)	40.25 (−5.96)
LFPT5	38.67 (−11.43)	42.26 (−7.43)
L2P	35.23 (−15.96)	34.63 (−16.86)
PP	29.41 (−5.79)	21.58 (−8.83)
O-LoRA	44.64 (−4.20)	42.83 (−9.11)
Any-SSR	**55.69** (−0.00)	**55.69** (−0.00)
Exemplar-free methods		
Replay (1%)	48.47 (−9.69)	47.04 (−10.24)
DER++ (1%)	49.22 (−8.32)	46.59 (−10.91)
SEEKR (1%)	54.99 (−2.61)	54.69 (−2.53)
Replay-based methods		
Upper-bound	59.38	

free method, SepFT, by 8.06%. This trend persists when evaluating with Order2. When comparing with replay-based methods, although replay-based methods take advantages of using exemplars and achieve considerable results compared with exemplar-free methods, Any-SSR still outperforms the state-of-the-art replay-based methods. Any-SSR achieves superior result compared with the state-of-the-art SEEKR with both Order1 and Order2. This leading pattern demonstrates the competitiveness of Any-SSR in adapting LLMs to downstream tasks. Furthermore, Any-SSR stands out as the sole method achieving 0 BWT, which showcases Any-SSR's ability to prevent interference between tasks. Notably, Any-SSR consistently produces identical results when learning with different orders, demonstrates Any-SSR's robustness and its invariance to task order. It is worth noting that we observed a few failed cases between NumGLUE-sm and NumGLUE-ds in TRACE, primarily due to ambiguous routing from shared prompts (e.g., "Solve the math problem with reasoning"). While this has minimal impact on general tasks (e.g., routine math problems), it may pose risks in critical domains like finance or healthcare, calling for targeted improvements.

4.4.2. Ablation study

Ablation on Components in Any-SSR. To provide a systematical analysis on each component in Any-SSR, we compare our method with two other strategies to demonstrate the effect of the isolated subspace training and analytic router. The alternative methods involve employing a single LoRA adapter for multi-task continual learning and utilizing a distinct LoRA adapter for each task while training the router using backpropagation (BP). For LoRA training, we set the rank as 8 while the learning rate as 1e-4. For BP-based router training, we utilize a two-layer MLP with the same expansion size E as AL-based router. As shown in Figure 3, our method demonstrates performance advantage over directly using a single LoRA and training a router with BP. For the methods of single LoRA, at each step, the new task can obtain competitive results while previous tasks encounter performance drop. This observation indicates the knowledge interference across different tasks. For the BP-based router, results display a more erratic pattern. To delve deeper into this phenomenon, we conducted additional experiments on these two routers.

AL Contributes to a Perfect Routing. To visualize the routing results of routers obtained with different methods, we report the routing accuracy at each step in Figure 4. For the BP-based router, it suffers progressive performance degradation (21.7% average accuracy drop per phase). Subsequently during the inference, the BP-based router can assign wrong subspaces to current task, leading to disrupted answer patterns for certain tasks and causing the performance of these tasks to drop to zero. In contrast, the analytic router showcases zero-forgetting throughout sequential training phases, maintaining 100% accuracy on previous tasks throughout the continual learning process. This stark difference highlights that the performance deterioration of the BP router stems from catastrophic forgetting. Our method benefits from the non-forgetting routers and achieves competitive results on all tasks.

Analysis on Hyperparameters in Any-SSR. As shown in 3, we present the performance of Any-SSR with different feature layer L_f and scale-up dimension E. Regarding L_f, lower feature layers (e.g., $L_f = 2$) exhibit non-zero BWT and lower OP, possibly due to the general nature of these lower features that compromising the router's accuracy. Performance initially improves with an increase in L_f, but starts to degrade beyond $L_f > 4$. This decline can be attributed to the decreased effectiveness of the remaining layers in task-specific learning with deeper feature layers, leading to a decrease in OP.Regarding E, it is noticeable that performance increases as E scales up from 5k to 10k when $L_f = 4$. However, further increasing E does not result in performance enhancements, indicating that choosing $E = 10k$ could be sufficient. The configuration of $L_f = 4, E = 10k$ dimensions yields optimal results.

4.4.3. General Ability after Continual Learning

To validate Any-SSR's performance of preserving general knowledge after continual learning in downstream tasks, we report the performance of models after CL on general skills benchmarks. It is crucial to emphasize that our Any-SSR utilizes only 1% of the merged validation subset data across all evaluated tasks exclusively for training the router module. This limited dataset is strictly employed to optimize the router's decision boundaries through constrained gradient updates, directing the general questions to the origin model

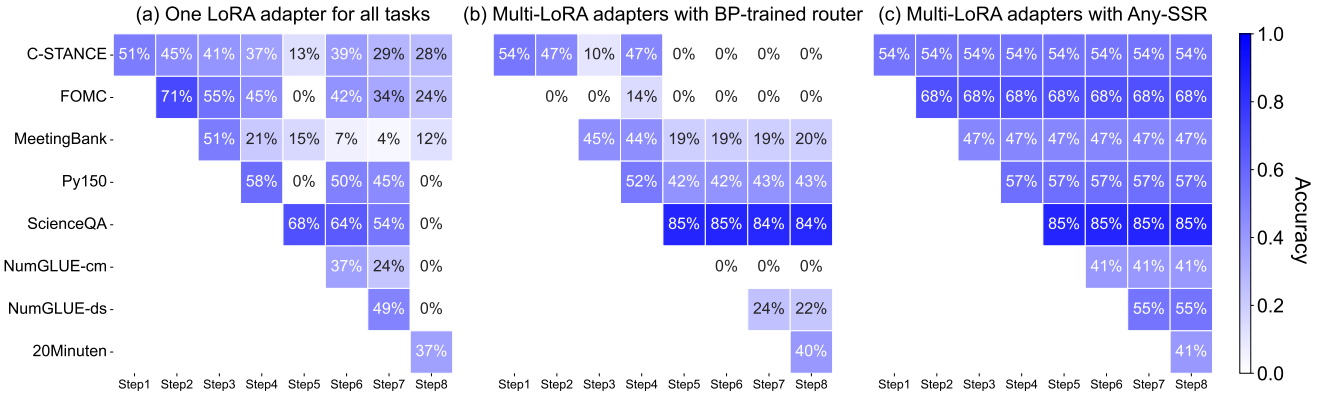


Figure 3. Heatmap comparing the accuracy of different methods across sequential tasks. Deep colors (blue) indicate higher accuracy rates, while light colors (white) represent lower performance.

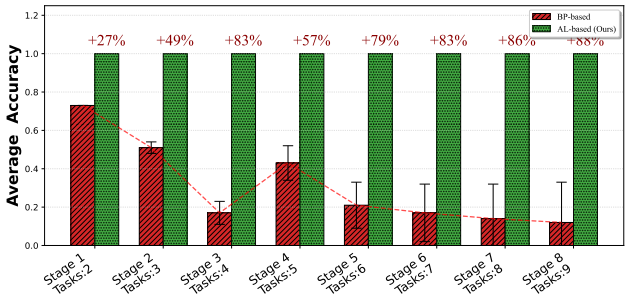


Figure 4. Comparison of the router's Acc in %. using Analytic Training and directly BP training. The horizontal axis denotes 8 incremental training stages (introduce a new task in each stage), and the vertical axis shows the average accuracy of all current tasks

Table 3. The performance (OP(BWT)) using different layers L_f and Feature Expanding size E on TRACE order1.

L_f	E	OP(BWT)
2	5000	54.76(-0.21)
2	10000	54.79(-0.19)
2	15000	54.79(-0.19)
4	5000	54.89(-0.12)
4	**10000**	**55.69(0.00)**
4	15000	55.69(0.00)
6	5000	53.21(0.00)
6	10000	53.21(0.00)
6	15000	53.21(0.00)

with parameters entirely isolated from any exposure to these samples.

As shown in Table 4, Any-SSR demonstrates relatively stable performance across various tasks. When compared with exemplar-free method SeqFT, Any-SSR can maintain better general skill. Although the replay-based methods (i.e., Replay and SEEKER) outperforms Any-SSR, the gaps are relative small. This observation demonstrates the effectiveness of Any-SSR to retain general knowledge while adhering to exemplar-free constrain.

Table 4. General abilities after continual learning with different methods with performance averaged over two task orders

	MMLU	GSM	BBH	TydiQA	BoolQ	PIQA	GA (DeltaGA)
LLaMA-2-7B-Chat	46.89	27.14	39.73	16.76	79.79	76.33	47.77
SeqFT	45.16	14.03	32.50	14.84	79.00	75.49	43.50 (-4.27)
Replay (1%)	45.49	12.70	33.46	14.65	78.69	75.65	43.44 (-4.33)
SEEKER (1%)	46.32	20.85	38.52	18.22	80.64	75.79	46.72 (-1.05)
Any-SSR	45.77	25.43	37.01	15.52	79.67	75.67	46.51 (-1.26)

5. Conclusion

Our proposed Any-SSR framework tackles catastrophic forgetting and computational inefficiency in continual learning through dynamic multi-task routing and parameter-efficient adaptation. Without storing or replaying historical data, it achieves near-perfect knowledge retention with theoretical guarantees while seamlessly integrating new tasks. Experiments demonstrate Any-SSR's robustness and efficiency, offering a reliable solution for continual learning in LLMs.

Acknowledgements

This research was supported by the AI Laboratory of Xiaomi Communications Co., Ltd., the National Natural Science Foundation of China (62306117), the Guangzhou Basic and Applied Basic Research Foundation (2024A04J3681), and GJYC program of Guangzhou (2024D03J0005).

References

[1] Eden Belouadah, Adrian Popescu, and Ioannis Kanellos. A comprehensive study of class incremental learning algorithms for visual tasks. *Neural Networks*, 135:38–54, 2021. 1

[2] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and SIMONE CALDERARA. Dark experience for general continual learning: a strong, simple baseline. In *Advances in Neural Information Processing Systems*, pages 15920–15930. Curran Associates, Inc., 2020. 6

[3] Francisco M. Castro, Manuel J. Marín-Jiménez, Nicolás Guil, Cordelia Schmid, and Karteek Alahari. End-to-end incremental learning. In *Computer Vision – ECCV 2018*, pages 241–257, Cham, 2018. Springer International Publishing. 2

[4] Arslan Chaudhry, Puneet K. Dokania, Thalaiyasingam Ajanthan, and Philip H. S. Torr. Riemannian walk for incremental learning: Understanding forgetting and intransigence. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018. 6

[5] Kexin Chen, Yuyang Du, Tao You, Mobarakol Islam, Ziyu Guo, Yueming Jin, Guangyong Chen, and Pheng-Ann Heng. Llm-assisted multi-teacher continual learning for visual question answering in robotic surgery. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10772–10778, 2024. 1

[6] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240): 1–113, 2023. 1

[7] Thomas M. Cover. Geometrical and statistical properties of systems of linear inequalities with applications in pattern recognition. *IEEE Transactions on Electronic Computers*, EC-14(3):326–334, 1965. 4

[8] Shihan Dou, Enyu Zhou, Yan Liu, Songyang Gao, Wei Shen, Limao Xiong, Yuhao Zhou, Xiao Wang, Zhiheng Xi, Xiaoran Fan, Shiliang Pu, Jiang Zhu, Rui Zheng, Tao Gui, Qi Zhang, and Xuanjing Huang. LoRAMoE: Alleviating world knowledge forgetting in large language models via MoE-style plugin. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1932–1945, Bangkok, Thailand, 2024. Association for Computational Linguistics. 3

[9] Shihan Dou, Enyu Zhou, Yan Liu, Songyang Gao, Wei Shen, Limao Xiong, Yuhao Zhou, Xiao Wang, Zhiheng Xi, Xiaoran Fan, Shiliang Pu, Jiang Zhu, Rui Zheng, Tao Gui, Qi Zhang, and Xuanjing Huang. LoRAMoE: Alleviating world knowledge forgetting in large language models via MoE-style plugin. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1932–1945, Bangkok, Thailand, 2024. Association for Computational Linguistics. 6

[10] Hao Fei, Yuan Yao, Zhuosheng Zhang, Fuxiao Liu, Ao Zhang, and Tat-Seng Chua. From multimodal LLM to human-level AI: Modality, instruction, reasoning, efficiency and beyond. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024): Tutorial Summaries*, pages 1–8, Torino, Italia, 2024. ELRA and ICCL. 1

[11] Team GLM. ChatGLM: A family of large language models from GLM-130b to GLM-4 all tools, 2024. 6

[12] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey. *Transactions on Machine Learning Research*, 2024. 1

[13] Jinghan He, Haiyun Guo, Kuan Zhu, Zihan Zhao, Ming Tang, and Jinqiao Wang. SEEKR: Selective attention-guided knowledge retention for continual learning of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 3254–3266, Miami, Florida, USA, 2024. Association for Computational Linguistics. 1, 3, 6

[14] Saihui Hou, Xinyu Pan, Chen Change Loy, Zilei Wang, and Dahua Lin. Learning a unified classifier incrementally via rebalancing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 2

[15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. 3

[16] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. 1, 3

[17] Nikitas Karanikolas, Eirini Manga, Nikoletta Samaridi, Eleni Tousidou, and Michael Vassilakopoulos. Large language models versus natural language understanding and generation. In *Proceedings of the 27th Pan-Hellenic Conference on Progress in Computing and Informatics*, page 278–290, New York, NY, USA, 2024. Association for Computing Machinery. 1

[18] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 2, 6

[19] Depeng Li and Zhigang Zeng. CRNet: A fast continual learning framework with random theory. *IEEE Transactions on Pat-*

tern Analysis and Machine Intelligence, 45(9):10731–10744, 2023. 2

[20] Qiwei Li, Yuxin Peng, and Jiahuan Zhou. FCS: Feature calibration and separation for non-exemplar class incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 28495–28504, 2024. 2

[21] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, 2018. 2, 6

[22] Haokun Liu, Yaonan Zhu, Kenji Kato, Atsushi Tsukahara, Izumi Kondo, Tadayoshi Aoyama, and Yasuhisa Hasegawa. Enhancing the LLM-based robot manipulation through human-robot collaboration. *IEEE Robotics and Automation Letters*, 9(8):6904–6911, 2024. 1

[23] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Lam Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *arXiv preprint arXiv:2110.07602*, 2021. 6

[24] Yaoyao Liu, Bernt Schiele, and Qianru Sun. Adaptive aggregation networks for class-incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2544–2553, 2021. 6

[25] David Lopez-Paz and Marc' Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017. 6

[26] Meryem M'hamdi and Jonathan May. Leitner-guided memory replay for cross-lingual continual learning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7808–7821, Mexico City, Mexico, 2024. Association for Computational Linguistics. 1

[27] OpenAI. GPT-4 technical report, 2024. 1

[28] Grégoire Petit, Adrian Popescu, Hugo Schindler, David Picard, and Bertrand Delezoide. FeTrIL: Feature translation for exemplar-free class-incremental learning. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 3911–3920, 2023. 2

[29] Fuli Qiao and Mehrdad Mahdavi. Learn more, but bother less: parameter efficient continual learning. In *Advances in Neural Information Processing Systems*, pages 97476–97498. Curran Associates, Inc., 2024. 3

[30] Chengwei Qin and Shafiq Joty. LFPT5: A unified framework for lifelong few-shot language learning based on prompt tuning of t5, 2022. 1, 6

[31] Anastasia Razdaibiedina, Yuning Mao, Rui Hou, Madian Khabsa, Mike Lewis, and Amjad Almahairi. Progressive prompts: Continual learning for language models, 2023. 6

[32] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. iCaRL: Incremental classifier and representation learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 2

[33] Weijieying Ren, Xinlong Li, Lei Wang, Tianxiang Zhao, and Wei Qin. Analyzing and reducing catastrophic forgetting in parameter efficient tuning, 2024. 3

[34] Wuxuan Shi and Mang Ye. Prototype reminiscence and augmented asymmetric knowledge aggregation for non-exemplar class-incremental learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1772–1781, 2023. 2

[35] Qwen Team. Qwen2.5: A party of foundation models, 2024. 6

[36] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023. 1, 6

[37] Fu-Yun Wang, Da-Wei Zhou, Han-Jia Ye, and De-Chuan Zhan. FOSTER: Feature boosting and compression for class-incremental learning. In *Computer Vision – ECCV 2022*, pages 398–414, Cham, 2022. Springer Nature Switzerland. 2

[38] Xiao Wang, Tianze Chen, Qiming Ge, Han Xia, Rong Bao, Rui Zheng, Qi Zhang, Tao Gui, and Xuanjing Huang. Orthogonal subspace learning for language model continual learning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10658–10671, Singapore, 2023. Association for Computational Linguistics. 1, 6

[39] Xiao Wang, Yuansen Zhang, Tianze Chen, Songyang Gao, Senjie Jin, Xianjun Yang, Zhiheng Xi, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, and Xuanjing Huang. Trace: A comprehensive benchmark for continual learning in large language models, 2023. 2, 5, 6

[40] Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. Learning to prompt for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 139–149, 2022. 6

[41] Max A Woodbury. *Inverting modified matrices*. Department of Statistics, Princeton University, 1950. 5

[42] Tongtong Wu, Linhao Luo, Yuan-Fang Li, Shirui Pan, Thuy-Trang Vu, and Gholamreza Haffari. Continual learning for large language models: A survey, 2024. 1

[43] Yang Yang, Zhiying Cui, Junjie Xu, Changhong Zhong, Wei-Shi Zheng, and Ruixuan Wang. Continual learning with bayesian model based on a fixed pre-trained feature extractor. *Visual Intelligence*, 1(1):5, 2023. 2

[44] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.10512*, 2023. 6

[45] Renrui Zhang, Dongzhi Jiang, Yichi Zhang, Haokun Lin, Ziyu Guo, Pengshuo Qiu, Aojun Zhou, Pan Lu, Kai-Wei Chang, Yu Qiao, Peng Gao, and Hongsheng Li. MATHVERSE: Does your multi-modal LLM truly see the diagrams in visual math problems? In *Computer Vision – ECCV 2024*, pages 169–186, Cham, 2025. Springer Nature Switzerland. 1

[46] Xuyang Zhao, Huiyuan Wang, Weiran Huang, and Wei Lin. A statistical theory of regularization-based continual learning. In *Forty-first International Conference on Machine Learning*, 2024. 2

[47] Bowen Zheng, Da-Wei Zhou, Han-Jia Ye, and De-Chuan Zhan. Multi-layer rehearsal feature augmentation for class-incremental learning. In *Forty-first International Conference on Machine Learning*, 2024. 2

[48] Fei Zhu, Xu-Yao Zhang, Chuang Wang, Fei Yin, and Cheng-Lin Liu. Prototype augmentation and self-supervision for incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5871–5880, 2021. 2

[49] Xudong Zhu, Yuanping Chen, and Xiaohui Rong. LHPF: An LLM-based hierarchical pipeline framework for spoken language understanding. In *2024 3rd International Conference on Electronics and Information Technology (EIT)*, pages 536–540, 2024. 1

[50] Huiping Zhuang, Zhenyu Weng, Hongxin Wei, Renchunzi Xie, Kar-Ann Toh, and Zhiping Lin. ACIL: Analytic class-incremental learning with absolute memorization and privacy protection. In *Advances in Neural Information Processing Systems*, pages 11602–11614. Curran Associates, Inc., 2022. 2

[51] Huiping Zhuang, Zhenyu Weng, Run He, Zhiping Lin, and Ziqian Zeng. GKEAL: Gaussian kernel embedded analytic learning for few-shot class incremental task. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7746–7755, 2023.

[52] Huiping Zhuang, Run He, Kai Tong, Ziqian Zeng, Cen Chen, and Zhiping Lin. DS-AL: A dual-stream analytic learning for exemplar-free class-incremental learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(15):17237–17244, 2024.

[53] Huiping Zhuang, Yuchen Liu, Run He, Kai Tong, Ziqian Zeng, Cen Chen, Yi Wang, and Lap-Pui Chau. F-OAL: Forward-only online analytic learning with fast training and low memory footprint in class incremental learning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. 1, 2